Architecture

Microservices architecture , MVVM .

Algorithms

Routing and sweet spot

- Dijkstra with cutoff (reverse isochrone)
- A* (point-to-point ETA)
- Contraction Hierarchies / Customizable CH
- RPHAST (one-to-many)
- Isochrone / concave hull

Geospatial

- H3 hexagonal indexing
- Geofencing (point-in-cell)
- Map matching (HMM / Viterbi)
- Kalman filter (GPS smoothing)
- Hysteresis (jitter control)

Dispatch and matching

- Hungarian algorithm (batch assignment)
- Bottleneck assignment (multi-scooter)
- Subset enumeration (6+4 coverage)
- Min-cost max-flow (alternative to Hungarian)

Placement and planning

- Maximum coverage / set cover (greedy)
- ILP (exact placement)
- Weighted demand coverage

Inventory and consistency

- Atomic reservation with TTL
- Row locking (`FOR UPDATE SKIP LOCKED`)
- Saga pattern (reserve, commit, rollback)
- Idempotency keys
- Fencing tokens
- Event sourcing / state machine

ETA and prediction

- Empirical p50/p90 per edge and time bucket
- Quantile regression
- Rolling percentile estimation

Verification

- Geofence + QR/BLE beacon check
- OTP verification
- Signed server timestamps (HMAC/JWT)

Tool stack

Area	Tool
Routing engine	OSRM / Valhalla / Google Routes API
Map data	OpenStreetMap
Spatial DB	PostgreSQL + PostGIS
Cell indexing	H3
Optimization	Google OR-Tools
Simulation	SimPy / custom discrete-event
Cache / live state	Redis
Message bus	Kafka / NATS
Realtime to client	WebSocket / FCM push
Observability	Prometheus, Grafana, OpenTelemetry
Maps (UI)	Mapbox / Google Maps SDK
Payments	Razorpay (auth and capture)

Tech Stack

Hybrid of flutter,kotlin and swift .

Words used in these docs

Term	Plain meaning
Gate	The exact entry point the customer picks for delivery
Gate pin	The gate's map location plus entry instructions
Sweet spot	The area a scooter can start from and still reach a gate within the time budget
Staging node	A legal place where a scooter waits for orders
Lot	One labelled batch of food with its prepared and expiry times
Reservation	A short hold on units so nobody else can buy them
Acceptance	The moment the server confirms the order and starts the 90 s clock
Leg	One scooter's share of an order

Term	Plain meaning
Hub	A small site where scooters restock
Kill switch	A control that turns the 90 s offer off for a zone
p90	The time that 9 out of 10 trips beat

What Each Algorithm Is For, and How Swiggy and Blinkit Work

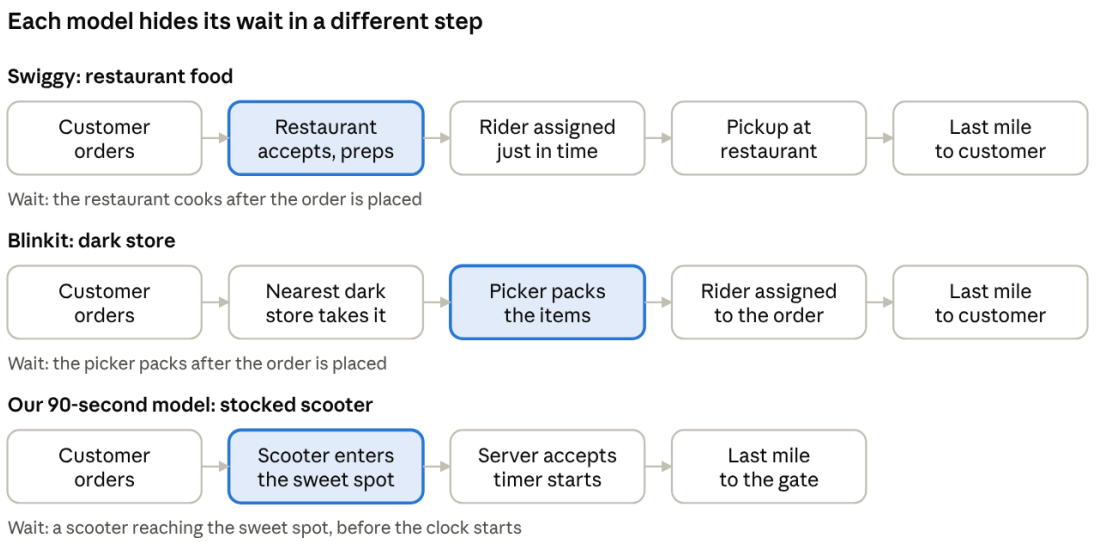
Algorithm map: which one solves which problem

Pick the algorithm by the question you are asking: how to get there, who is near, who gets the order, where scooters wait, or how stock stays correct.

Question	Algorithm	What it does	Where we use it
How to get there	Dijkstra	Shortest path from one point to every point within a cost limit	Reverse isochrone from the gate, cut off at 55 s
How to get there	A*	Shortest path between two points, guided by a distance estimate	Final live ETA check before acceptance
How to get there	Contraction Hierarchies	Preprocesses the road graph so routes answer in milliseconds	Inside OSRM / Valhalla
How to get there	RPHAST	Many-destination travel times from one source, fast	Scooters-to-gate time matrix
How to get there	Concave hull	Draws the outline of a reachable area	Sweet-spot polygon on the ops map
Who is near	H3 hex indexing	Turns lat/long into a cell id for O(1) lookup	Sweet spot stored as a set of cells
Who is near	Geofencing	Tests whether a point is inside an area	Fires when a scooter enters the sweet spot
Who is near	Map matching (HMM / Viterbi)	Snaps noisy GPS to the most likely road	Scooter position
Who is near	Kalman filter	Smooths noisy readings	GPS position and speed
Who is near	Hysteresis	Needs repeated evidence before flipping state	Two pings in-zone before trigger
Who gets the order	Hungarian algorithm	Best one-to-one pairing of orders and scooters	Batch dispatch every 1-2 s

Question	Algorithm	What it does	Where we use it
Who gets the order	Min-cost max-flow	Same pairing with capacities on each side	Alternative to Hungarian
Who gets the order	Bottleneck assignment	Minimises the slowest leg	6+4 orders, where the slowest scooter sets the time
Who gets the order	Subset enumeration	Tries every small combination	Picks 2-3 scooters that cover 10 units
Where scooters wait	Set cover / max coverage	Picks few locations that cover most demand	Staging-node selection
Where scooters wait	ILP	Exact optimum of the same problem	Final placement (OR-Tools)
Stock stays correct	Reservation with TTL	Holds units briefly, then releases them	Stops two customers buying the last burger
Stock stays correct	FOR UPDATE SKIP LOCKED	Lets workers take different rows without waiting	Concurrent reservations
Stock stays correct	Saga	Multi-step change with undo steps	Reserve, prepare, commit, roll back
Stock stays correct	Idempotency keys	A retried request runs once	Payment and acceptance
Stock stays correct	Fencing tokens	Rejects writes from a stale holder	Old rider device cannot double-commit
Stock stays correct	Event sourcing / state machine	Every order change is a logged event	Order lifecycle and audit
How long it takes	p50/p90 per edge and hour	Observed travel-time percentiles	ETA lookup table
How long it takes	Quantile regression	Predicts a percentile from features	ETA once data exists
Proof	Signed server timestamps	Time that the rider app cannot alter	Acceptance and arrival times

How an order flows



order flow comparison · 3 models

Swiggy waits for the kitchen and Blinkit for the picker, both after the order; our model waits for the scooter to reach the sweet spot, then starts the clock. Swiggy's flow follows its just-in-time assignment, and Blinkit's the dark-store model, both as described in the sources below.

Architecture they run

Both platforms are microservices around an order state machine, a dispatch engine and live-location streaming; Blinkit adds a dark-store layer for stock and picking. Swiggy's own engineering blog says hundreds of services take part in fulfilling one order. Neither company publishes a full architecture, so this table mixes primary posts with agency and case-study descriptions, marked in the last column.

Component	Role	Reported for	Evidence
Customer, restaurant and rider apps	Order, accept and prepare, pick up and deliver	Swiggy	Agency guide (typical build)
Picker app	Dark-store staff collect and pack items	Blinkit-style quick commerce	Agency guide (typical build)
Order service	Moves an order through Received, Confirmed, Preparing, Assigned, Picked up, Delivered	Swiggy	Agency guide (typical build)
Dispatch / assignment engine	Pairs orders with riders, holds riders until food is nearly ready	Swiggy	Case study

Component	Role	Reported for	Evidence
ETA service	Predicts delivery time at cart and after the order	Swiggy	Engineering blog, via summary
Data science platform	Serves ML predictions with a batch API under a latency limit	Swiggy	Vendor case study
Event streaming	Orders, GPS pings and SLA predictions flow as events	Swiggy	Conference talk
Search	Multi-cluster OpenSearch for catalogue search	Blinkit	Conference session
Dark-store layer	Per-store stock, picking and packing; orders routed to the nearest store	Blinkit	Case studies, agency guides

Tech stack and tools they use

The confirmed pieces are Kafka, an Akka-based ML serving platform and OpenSearch; the rest of what circulates online is typical-clone advice, not confirmed internals.

Layer	Reported	Company	Evidence
Event streaming	Apache Kafka, moved from self-managed open source to Confluent-managed Kafka	Swiggy	Conference talk and interview
ML serving	Akka-based Data Science Platform with batch prediction; vendor reports P99 latency cut from 144 ms to 71 ms at 5,000+ predictions per second	Swiggy	Vendor case study
ETA models	Started with gradient boosted trees, later a multi-output deep learning model	Swiggy	Secondary summary of engineering blog
Routing	In-house route estimation, because commercial map APIs were slow and costly at their volume	Swiggy	Secondary summary of engineering blog
Search	OpenSearch, multi-cluster	Blinkit	Conference session
Quick-commerce data platform	Kafka, Spark on Databricks, Flink	Company not confirmed	Conference session listing
Typical clone stack	React Native or Flutter, Node.js / Python / Go, PostgreSQL or MongoDB, Redis, Socket.io or Firebase	Neither	Agency blogs, not confirmed

Algorithms they use

Their core problem is assigning riders without wasting waiting time, and predicting delivery time well enough to promise it.

Problem	Technique reported	Company	Evidence
Order-to-rider assignment	Scores every order-and-rider pairing with ML models (ETA, distance, availability) and commits within about a minute	Swiggy	Vendor case study
Idle waiting at restaurants	Just-in-time assignment: a rider is earmarked near the restaurant but dispatched when food is almost ready	Swiggy	Case study
Delivery time model	Delivery time = max(assignment delay + first mile, prep time) + last mile	Swiggy	Secondary summary of engineering blog
ETA prediction	Gradient boosted trees, then multi-input multi-output deep learning with entity embeddings	Swiggy	Secondary summary of engineering blog
Batching	Two nearby orders to one rider, only if combined delivery time stays under a limit	Typical	Agency guide
Demand to dispatch	Demand prediction, inventory allocation, dark-store network, automated dispatch	Blinkit	Case study
Inventory accuracy	Atomic stock decrement in Redis	Typical quick-commerce build	Agency guide

APIs

These apps lean on a handful of external APIs, and our system needs its own internal API surface; the endpoints below are my proposal, not Swiggy's or Blinkit's.

External APIs (typical for this kind of app)

Need	Typical API
Maps, geocoding, turn-by-turn	Google Maps SDK / Routes API or Mapbox
Payments (authorize, capture, refund)	Razorpay or similar
Push notifications	Firebase Cloud Messaging
OTP and SMS	An SMS gateway
Live location	WebSocket or Firebase Realtime Database

Proposed internal APIs for the 90-second system

Caller	Endpoint	Purpose
Customer	`POST /v1/gates`	Save the delivery gate pin
Customer	`GET /v1/availability?gate_id=`	Items and quantities currently within reach
Customer	`POST /v1/orders`	Create the order; state becomes MATCHING
Customer	`GET /v1/orders/{id}`	Order state and countdown
Customer	`WS /v1/orders/{id}/stream`	Live status events
Customer	`POST /v1/orders/{id}/cancel`	Cancel before acceptance
Rider	`POST /v1/riders/me/location`	GPS ping
Rider	`POST /v1/riders/me/stock/scan`	Verify stock and lot
Rider	`POST /v1/offers/{id}/ready`	Packed and ready
Rider	`POST /v1/orders/{id}/arrive`	Arrival evidence
Rider	`POST /v1/orders/{id}/handover`	Quantity handed over, OTP
Internal	`POST /internal/reservations`	Hold stock with a TTL
Internal	`POST /internal/orders/{id}/commit`	Single acceptance, idempotency key required
Internal	`GET /internal/sweetspot/{gate_id}`	Cell set and travel times for a gate
Ops	`POST /ops/lots/{id}/quarantine`	Pull a lot with a temperature or expiry problem

What we copy and what we change

We copy their event backbone and dispatch discipline, and replace the parts built around restaurants and warehouses.

Area	Copy from Swiggy / Blinkit	Change for our model
Events	Kafka-style stream for orders, GPS and stock	Same
Dispatch	Just-in-time idea: hold the rider until the moment is right	Trigger is the sweet-spot entry, not food-ready time
Supply	Dark-store stock per area	Stock lives inside the scooter, tracked by lot and temperature
ETA	Start with simple rules, add ML when data exists	Compare p90 travel time against the 90 s budget

Area	Copy from Swiggy / Blinkit	Change for our model
Picking	Picker app at the dark store	Rider packs while stationary, before acceptance
Search	Catalogue search	Not needed for a 1-3 item menu

Sources

Read through search excerpts as of 23 October 2026; I did not open every page in full, so treat figures as reported, not verified.

- Swiggy engineering blog: engineering challenges
- Akka: Swiggy customer story
- Lightbend: Swiggy Data Science Platform latency
- Confluent Current 2025: Swiggy talk on Kafka
- Frontier Enterprise: Swiggy's platform evolution
- ICMR case study: Swiggy's delivery model
- MarkHub24: Swiggy delivery time prediction (summary of Swiggy blog)
- OpenSearchCon India 2026: Blinkit's multi-cluster OpenSearch
- Cognitute: Blinkit dark store network and logistics
- RaftLabs: building an app like Swiggy
- RaftLabs: building a quick-commerce app like Blinkit